МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по практической работе №2

по дисциплине

«Стили и методы программирования. Часть 2 (МДН)»

на тему

«Разработка логических программ на языке Prolog для работы с базами знаний и решения задач поиска»

Выполнил:
Лебедевич Артем Владимирович
магистрант 2 курса группы 556301

Преподаватель:
Парамонов Антон Иванович
кандидат технических наук
доцент

Дата: 11.10.2026

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 2

2 Используемое ПО 3

3 Ход выполнения работы 5
3.1 Создание проекта 5
3.2 Задание 2: динамическая база фактов об отношении «больше» 5
3.3 Задание 3: база знаний «Друзья» (вариант 3) 11
3.4 Задание 4: логическая задача «Спортсмены» (вариант 3) 15

4 Анализ результатов 19

5 Выводы 20

Приложение А. Контрольные вопросы для Moodle 21

1 ЦЕЛЬ РАБОТЫ

Изучить базовый синтаксис и семантику языка Prolog, освоить методы создания и модификации динамических баз фактов и правил, научиться решать задачи логического вывода и поиска с ограничениями.

2 ИСПОЛЬЗУЕМОЕ ПО

Версия интерпретатора и параметры программной среды приведены в таблице 2.1; результат запроса версии показан на рисунке 2.1.

Таблица 2.1 – Программная среда

Параметр	Значение
Интерпретатор Prolog	SWI-Prolog 10.0.2 (threaded, 64 bits), сборка `arm64-darwin`
Способ установки	менеджер пакетов Homebrew, формула `swi-prolog`
Операционная система	macOS 26.6.2 (сборка 25G83)
Аппаратная платформа	Apple M4 Pro, архитектура arm64
Интерпретатор	`/opt/homebrew/bin/swipl`, консольный top-level в терминале zsh
Редактор	Visual Studio Code
Кодировка исходных файлов	UTF-8; атомы и имена предикатов записаны кириллицей
Рабочая директория	`~/Projects/prolog/lab_02`

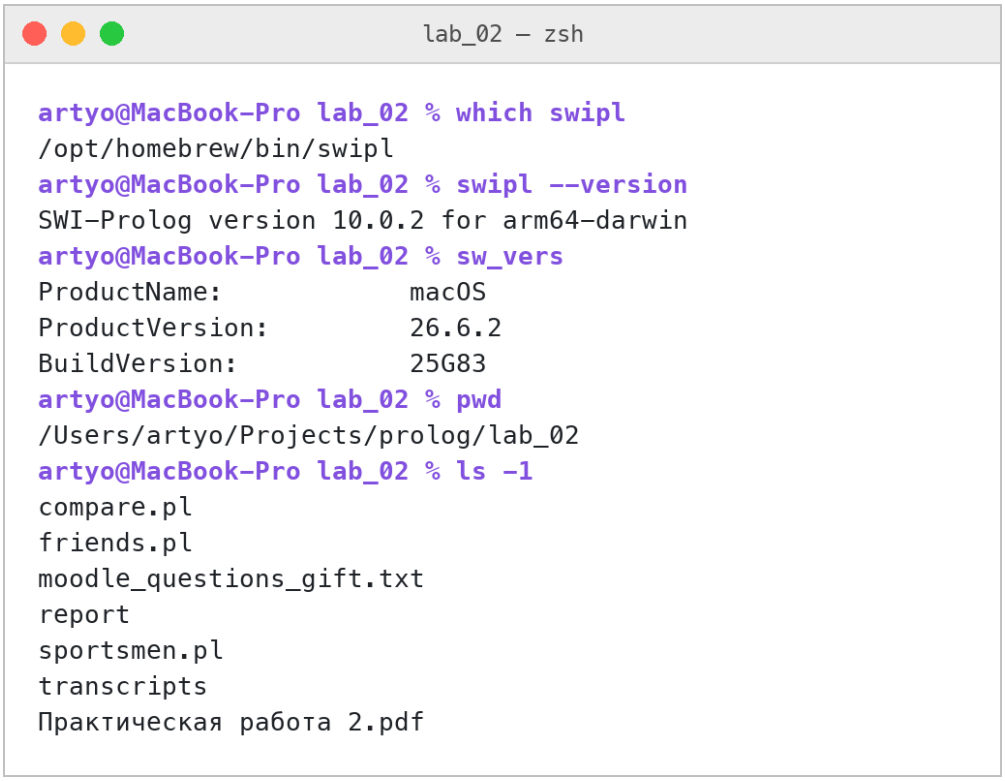


Рисунок 2.1 – Версия интерпретатора, ОС и содержимое рабочей директории проекта

Обоснование выбора. Использован тот же SWI-Prolog, что и в практической работе 1: он соответствует стандарту ISO, содержит нужные для работы встроенные средства (динамические предикаты `assertz/retract`, ввод строк, `findall/3`, `permutation/2`) и корректно работает с кириллицей в атомах, что позволяет записывать базы знаний варианта в том виде, в котором они даны в задании.

3 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

3.1 Создание проекта

Новый проект — директория `lab_02` внутри репозитория `prolog`. Каждое задание оформлено отдельным файлом с программой (таблица 3.1); интерпретатор запускается из этой директории, поэтому файлы загружаются по короткому имени: `swipl compare.pl` либо `?- [compare].`

Таблица 3.1 – Состав проекта

Задание	Файл	Содержание
2	`compare.pl`	динамическая база отношений «больше», ввод с консоли, поиск наибольшего и наименьшего объекта
3	`friends.pl`	база знаний «Друзья» (вариант 3) и ответы на её вопросы
4	`sportsmen.pl`	логическая задача про спортсменов (вариант 3)

3.2 Задание 2: динамическая база фактов об отношении «больше»

Постановка. С консоли вводятся отношения между объектами, например: «яблоко больше сливы», «авокадо меньше папайи». Число объектов заранее неизвестно, ввод продолжается до заданного символа. По введённым фактам нужно определить самый большой и самый маленький объекты.

Решение. Программа `lab_02/compare.pl` приведена ниже. Отношение вводится строкой вида `A > B` (A больше B) или `A < B` (A меньше B); допустимы и слова «больше», «меньше». Символом остановки служит `#`, он задан фактом `stop_symbol/1`.

```
% ============================================
% Динамическая база фактов об отношении «больше»
% SWI-Prolog
% --------------------------------------------
% Практическая работа №2, задание 2, «Стили и
% методы программирования. Часть 2».
%
% Пользователь вводит с консоли отношения вида
%     яблоко > слива        (яблоко больше сливы)
%     вишня < слива         (вишня меньше сливы)
% до символа остановки '#'. Факты запоминаются в
% динамической базе, после чего программа выводит
% самый большой и самый маленький объект.
%
% Загрузка:  swipl compare.pl
% Запуск:    ?- start.
```

```
% ========================================

% --- Динамическая база фактов ---
% bigger(X, Y): X больше Y (введённые пользователем факты).
% object(X):    X встретился во вводе (порядок сохраняется).
:- dynamic bigger/2.
:- dynamic object/1.

% Символ, которым завершается ввод.
stop_symbol('#').

% --- Правила вывода ---
% greater(X, Y): X больше Y — прямо или по цепочке (транзитивное
 ↪  замыкание).
% Цикл невозможен: add_relation/2 не принимает противоречивых фактов.
greater(X, Y) :- bigger(X, Y).
greater(X, Y) :- bigger(X, Z), greater(Z, Y).

% largest(X): X самый большой — ни один объект не больше X.
largest(X) :- object(X), \+ bigger(_, X).

% smallest(X): X самый маленький — X не больше ни одного объекта.
smallest(X) :- object(X), \+ bigger(X, _).

% --- Ввод с консоли ---
% start: очистить базу, считать отношения, вывести ответ.
start :-
    retractall(bigger(_, _)),
    retractall(object(_)),
    stop_symbol(Stop),
    format("Вводите отношения: «A > B» (A больше B) или «A < B» (A меньше
     ↪  B).~n"),
    format("Для завершения ввода введите ~w~n", [Stop]),
    prompt(Old, ''),                % убрать стандартное приглашение «|: »
    read_relations(Stop),
    prompt(_, Old),
    report.

% read_relations(+Stop): читать строки, пока не встретится символ остановки
% или конец ввода (Ctrl+D).
read_relations(Stop) :-
    format("ввод: "),
    flush_output,
    read_line_to_string(user_input, Line),
    (   Line == end_of_file
    ->  nl
    ;   normalize_space(string(Clean), Line),
        (   atom_string(Stop, Clean)
        ->  true
        ;   process_line(Clean),
            read_relations(Stop)
        )
```

```
    ).

% process_line(+Строка): разобрать строку «A знак B» и добавить факт.
process_line("") :- !.
process_line(Line) :-
    string_lower(Line, Lower),
    split_string(Lower, " ", "", [A, Sign, B]),
    sign_direction(Sign, Dir),
    !,
    atom_string(X, A),
    atom_string(Y, B),
    (   Dir == bigger
    ->  add_relation(X, Y)
    ;   add_relation(Y, X)
    ).
process_line(Line) :-
    format("  не понял «~s»: нужен формат «A > B» или «A < B»~n", [Line]).

sign_direction(">", bigger).
sign_direction("<", smaller).
sign_direction("больше", bigger).
sign_direction("меньше", smaller).

% add_relation(+X, +Y): запомнить «X больше Y», если факт не противоречит базе.
add_relation(X, X) :-
    !,
    format("  отклонено: ~w не может быть больше самого себя~n", [X]).
add_relation(X, Y) :-
    greater(X, Y),
    !,
    format("  уже известно: ~w больше ~w~n", [X, Y]).
add_relation(X, Y) :-
    greater(Y, X),
    !,
    format("  отклонено: противоречит базе (~w больше ~w)~n", [Y, X]).
add_relation(X, Y) :-
    assertz(bigger(X, Y)),
    remember(X),
    remember(Y),
    format("  принято: ~w больше ~w~n", [X, Y]).

remember(X) :- object(X), !.
remember(X) :- assertz(object(X)).

% --- Ответ на контрольный вопрос ---
report :-
    findall(X, object(X), Objects),
    findall(X, largest(X), Max),
    findall(X, smallest(X), Min),
    length(Objects, N),
    aggregate_all(count, bigger(_, _), M),
```

```
    nl,
    format("Объектов: ~d, введённых отношений: ~d~n", [N, M]),
    answer("Самый большой", Max),
    answer("Самый маленький", Min).

answer(_, []) :-
    !,
    format("Отношения не введены.~n").
answer(Title, [X]) :-
    !,
    format("~s: ~w~n", [Title, X]).
answer(Title, Xs) :-
    atomic_list_concat(Xs, ', ', List),
    format("~s: единого нет, между собой несравнимы: ~w~n", [Title, List]).
```

Основная логика.

1. **Динамическая база.** Директивы `:- dynamic` объявляют предикаты `bigger/2` (факт «X больше Y») и `object/1` (объект встретился во вводе) изменяемыми во время работы. Факты добавляются вызовом `assertz/1` в порядке ввода; в начале `start/0` база очищается через `retractall/1`, поэтому программу можно запускать повторно.
2. **Ввод.** Предикат `read_relations/1` рекурсивно читает строки `read_line_to_string/2` до символа остановки или конца ввода (Ctrl+D). Рекурсия хвостовая, число строк не ограничено. Стандартное приглашение «`|: `» убрано вызовом `prompt/2`, вместо него печатается своё, поэтому `flush_output/0` вызывается до чтения. Строка нормализуется (`normalize_space/2`), приводится к нижнему регистру и разбивается `split_string/4` на три части: два объекта и знак.
3. **Контроль данных.** Предикат `add_relation/2` отклоняет отношение объекта к самому себе и факт, противоречащий уже известным (если Y больше X прямо или по цепочке), а уже выводимый факт не дублирует. Без этой проверки цикл («a больше b», «b больше a») привёл бы к бесконечной рекурсии в `greater/2`. Строка неверного формата сообщается и пропускается.
4. **Правила вывода.** `greater/2` — транзитивное замыкание: базовый случай — прямой факт `bigger`, рекурсивный шаг — цепочка через промежуточный объект. Правило `largest(X)` истинно для объекта, которого не превосходит ни один другой (`\+ bigger(_, X)`); `smallest(X)` — для объекта, который не больше никакого.
5. **Ответ.** `report/0` собирает решения `largest/1` и `smallest/1` в списки через `findall/3`. Если в списке один объект, он и есть ответ; если несколько, единого наибольшего (наименьшего) нет — объекты принадлежат разным цепочкам и между собой несравнимы, о чём выводится сообщение.

Результаты. На рисунке 3.1 введён пример из задания: «яблоко больше сливы, авокадо больше сливы, но меньше папайи, дуриан больше папайи, вишня меньше сливы». Принятые отношения нормализованы к виду «больше» («авокадо меньше папайи» → «папайя больше авокадо»). Команда `listing(bigger/2)` показывает, что база действительно построена во время работы программы, а `greater(дуриан, вишня)` выводит факт, которого в базе нет: дуриан > папайя > авокадо > слива > вишня.

Самый маленький объект — вишня. Единого самого большого нет: цепочка `дуриан > папайя > авокадо > слива > вишня` и отношение `яблоко > слива` не позволяют сравнить яблоко с дурианом, поэтому программа выдаёт оба максимальных объекта. Ответ «дуриан» появляется, если добавить недостающее условие «яблоко меньше дуриана» (рисунок 3.2).

lab_02 — swipl compare.pl

```
artyo@MacBook-Pro lab_02 % swipl compare.pl

1 ?- start.
Вводите отношения: «A > B» (A больше B) или «A < B» (A меньше B).
Для завершения ввода введите #
ввод: яблоко > слива
  принято: яблоко больше слива
ввод: авокадо > слива
  принято: авокадо больше слива
ввод: авокадо < папайя
  принято: папайя больше авокадо
ввод: дуриан > папайя
  принято: дуриан больше папайя
ввод: вишня < слива
  принято: слива больше вишня
ввод: #

Объектов: 6, введённых отношений: 5
Самый большой: единого нет, между собой несравнимы: яблоко, дуриан
Самый маленький: вишня
true.

2 ?- listing(bigger/2).
:- dynamic bigger/2.

bigger(яблоко, слива).
bigger(авокадо, слива).
bigger(папайя, авокадо).
bigger(дуриан, папайя).
bigger(слива, вишня).

true.

3 ?- greater(дуриан, вишня).
true ;
false.

4 ?- halt.
```

Рисунок 3.1 – Пример из задания: ввод отношений, ответ, содержимое динамической базы

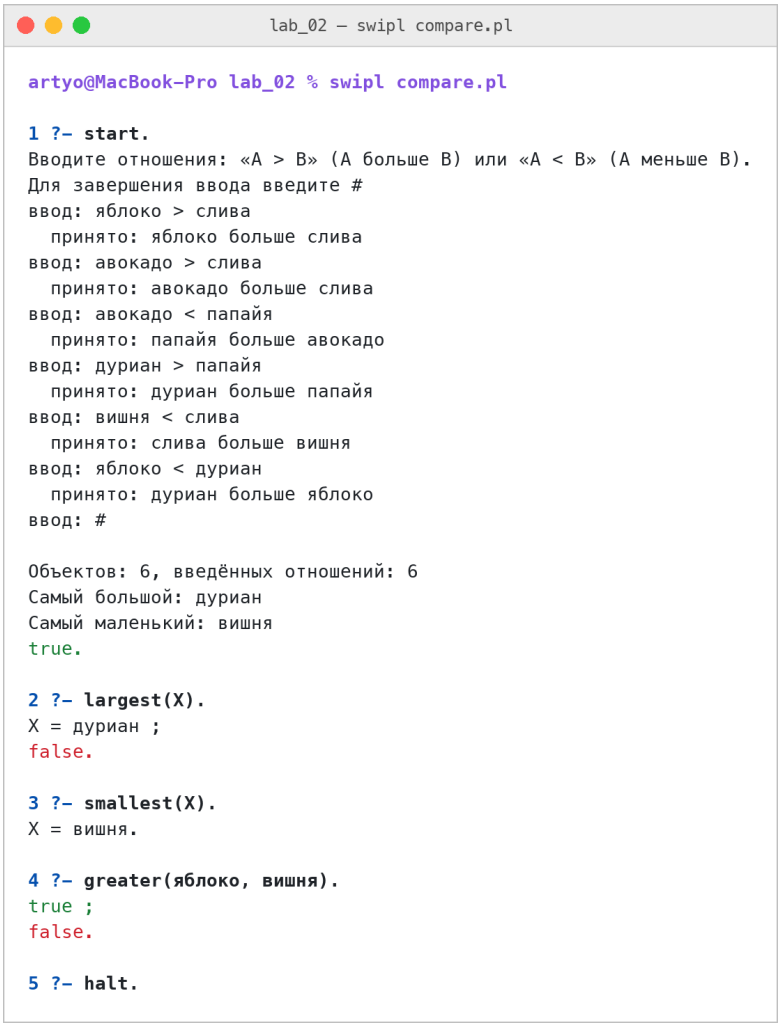


Рисунок 3.2 – Тот же ввод с дополнительным условием «яблоко меньше дуриана»

Работа с некорректными данными показана на рисунке 3.3: противоречие, сравнение объекта с самим собой, неизвестный знак, пустая строка, повтор и факт, уже выводимый из известных.

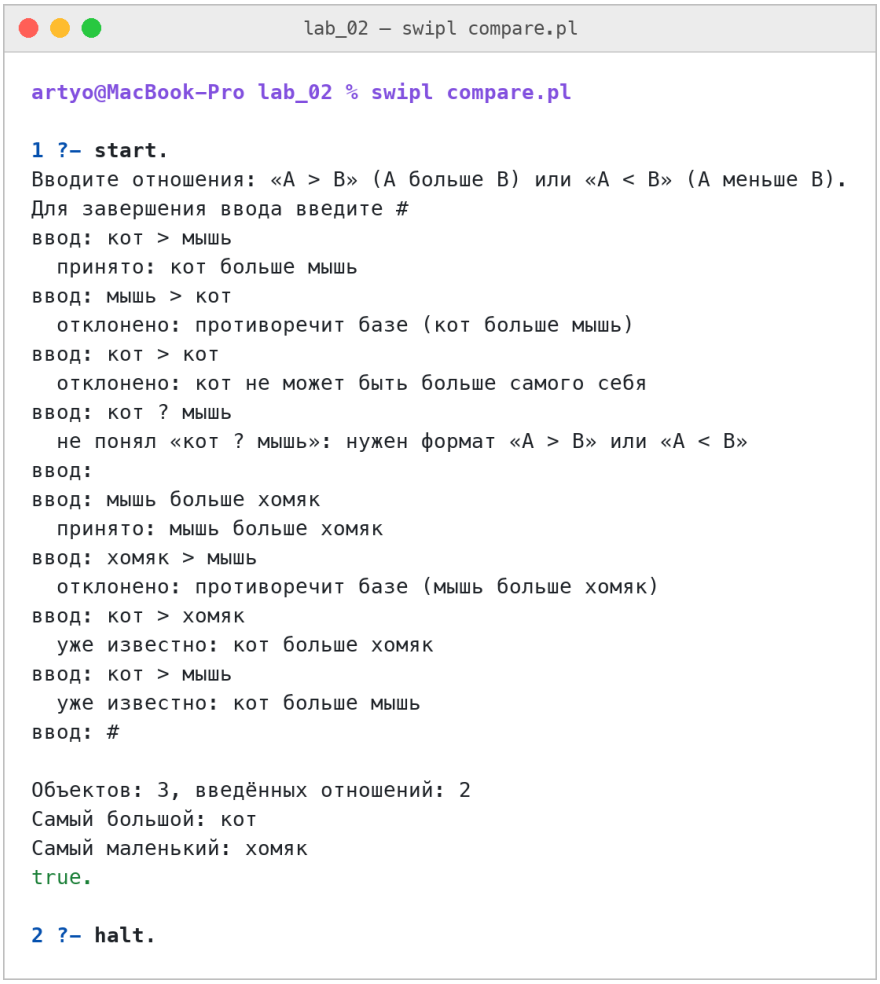


Рисунок 3.3 – Обработка некорректного и избыточного ввода

3.3 Задание 3: база знаний «Друзья» (вариант 3)

Постановка. Написать программу для базы знаний «Друзья» и ответить на вопросы: кто родился в 1988 году; кто родился в мае; у кого хобби книги; какое хобби у Маши; у кого хобби и кино, и спорт; вывести список фамилий и имён всех друзей.

Решение. Программа `lab_02/friends.pl`. Факты `день_рождения/5` и `хобби/3` записаны в точности как в задании. Поверх них добавлены правила: `друг/2` (каждый друг указан днём рождения), `родился_в_году/3`, `родился_в_месяце/3` и `увлекается_обоими/4`. Предикат `ответы/0` печатает ответы на все вопросы; вспомогательный `вывести/3` собирает все решения цели через `findall/3` и печатает их по одному.

```
% ============================================
% База знаний «Друзья»
% SWI-Prolog
% --------------------------------------------
% Практическая работа №2, задание 3 (вариант 3),
% «Стили и методы программирования. Часть 2».
%
```

```
% Загрузка:  swipl friends.pl
%       или  ?- [friends].
% ============================================

% --- Факты ---
% день_рождения(Фамилия, Имя, День, Месяц, Год).
день_рождения(иванова, лена, 12, июнь, 1988).
день_рождения(сидоренко, вася, 25, март, 1987).
день_рождения(сидоренко, маша, 6, май, 1988).

% хобби(Фамилия, Имя, Хобби).
хобби(иванова, лена, книги).
хобби(сидоренко, маша, танцы).
хобби(сидоренко, вася, спорт).
хобби(сидоренко, вася, кино).
хобби(сидоренко, маша, кино).

% --- Правила ---
% друг(Ф, И): человек с фамилией Ф и именем И — друг;
% каждый друг указан в базе днём рождения.
друг(Ф, И) :- день_рождения(Ф, И, _, _, _).

% родился_в_году(Ф, И, Год): друг Ф И родился в году Год.
родился_в_году(Ф, И, Год) :- день_рождения(Ф, И, _, _, Год).

% родился_в_месяце(Ф, И, Месяц): друг Ф И родился в месяце Месяц.
родился_в_месяце(Ф, И, Месяц) :- день_рождения(Ф, И, _, Месяц, _).

% увлекается_обоими(Ф, И, Х1, Х2): у друга Ф И есть оба хобби — Х1 и Х2.
увлекается_обоими(Ф, И, Х1, Х2) :- хобби(Ф, И, Х1), хобби(Ф, И, Х2).

% --- Контрольные вопросы варианта ---
% ответы: печатает ответы на все вопросы подряд.
ответы :-
    вывести("Кто родился в 1988 году?",
            родился_в_году(Ф, И, 1988), Ф-И),
    вывести("Кто родился в мае?",
            родился_в_месяце(Ф, И, май), Ф-И),
    вывести("У кого хобби книги?",
            хобби(Ф, И, книги), Ф-И),
    вывести("Какое хобби у Маши?",
            хобби(_, маша, Х), Х),
    вывести("У кого хобби и кино, и спорт?",
            увлекается_обоими(Ф, И, кино, спорт), Ф-И),
    вывести("Фамилии и имена всех друзей:",
            друг(Ф, И), Ф-И).

% вывести(+Вопрос, :Цель, +Шаблон): напечатать вопрос и все решения Цели.
% Шаблон показывает, какие переменные Цели составляют ответ.
вывести(Вопрос, Цель, Шаблон) :-
    findall(Шаблон, Цель, Ответы),
    format("~s~n", [Вопрос]),
```

```
60      (   Ответы == []
61      ->  format("  никто~n")
62      ;   forall(member(O, Ответы), format("  ~w~n", [O]))
63      ).
```

Основная логика. Вопросы с одним условием — это запросы с анонимной переменной на неинтересных позициях: `день_рождения(Ф, И, _, _, 1988)`. Вопрос «и кино, и спорт» выражается конъюнкцией двух целей с общими переменными `Ф` и `И`: они заставляют оба хобби принадлежать одному человеку. Вопрос о Маше задаёт имя константой, а фамилию оставляет анонимной.

Запросы вводились в top-level (рисунок 3.4); перебор решений выполнялся нажатием `;`. Сводка ответов приведена в таблице 3.2.

Таблица 3.2 – Вопросы варианта 3 к базе знаний «Друзья»

№	Вопрос	Запрос	Ответ
1	Кто родился в 1988 году?	`день_рождения(Ф, И, _, _, 1988)`	Лена Иванова, Маша Сидоренко
2	Кто родился в мае?	`день_рождения(Ф, И, _, май, _)`	Маша Сидоренко
3	У кого хобби книги?	`хобби(Ф, И, книги)`	Лена Иванова
4	Какое хобби у Маши?	`хобби(_, маша, Х)`	танцы, кино
5	У кого хобби и кино, и спорт?	`хобби(Ф, И, кино), хобби(Ф, И, спорт)`	Вася Сидоренко
6	Список всех друзей	`друг(Ф, И)`	Иванова Лена, Сидоренко Вася, Сидоренко Маша



Рисунок 3.4 – Ответы на вопросы варианта 3 в интерактивном режиме

Те же ответы выводит предикат `ответы/0`; запрос с `findall/3` собирает список друзей одним значением (рисунок 3.5).

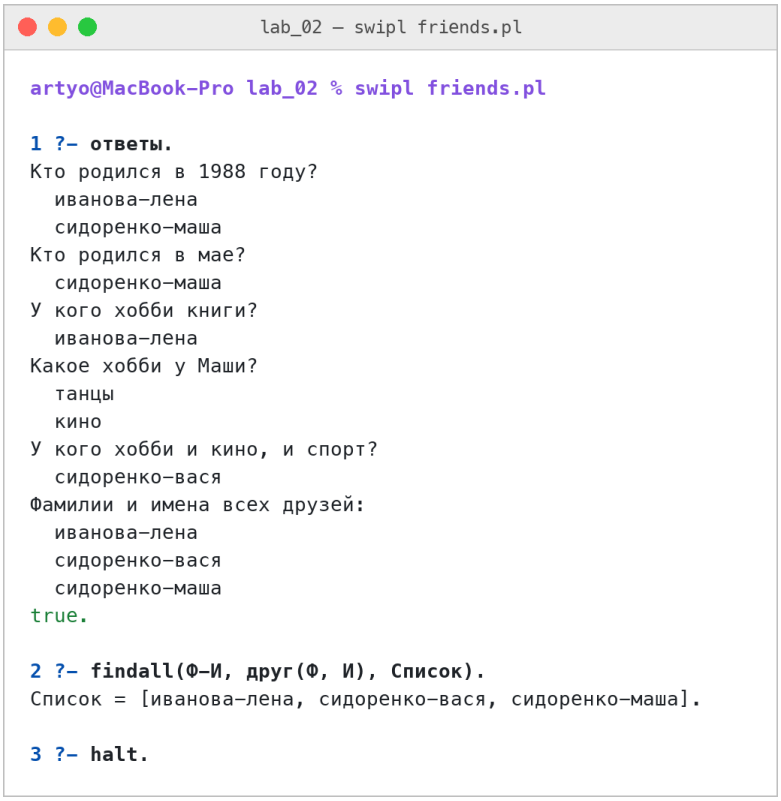


Рисунок 3.5 – Результат работы предиката `ответы/0` и список друзей одним термом

3.4 Задание 4: логическая задача «Спортсмены» (вариант 3)

Условие. Тонни, Майкл и Джон — спортсмены. Один из них боксёр, другой футболист, третий пловец. Один любит снег, другой — дождь, третий — солнечную погоду. Тот, кто любит солнечную погоду, — не боксёр. Тонни не любит снег. Майкл не любит солнечную погоду. Футболист любит снег. Тонни не пловец. Определить, у кого какая любимая погода и кто каким спортом занимается.

Формализация. Каждый спортсмен описывается термом `человек(Имя, Спорт, Погода)`. Три спортсмена образуют список, в котором виды спорта и виды погоды — перестановки трёх возможных значений: так выполняется условие «один — боксёр, другой — футболист, третий — пловец» (и то же для погоды). Остальные условия записаны как ограничения (таблица 3.3).

Таблица 3.3 – Запись условий задачи на Прологе

№	Условие	Цель
1	Любящий солнце не боксёр	`\+ member(человек(_, боксёр, солнце), Люди)`
2	Тонни не любит снег	`\+ member(человек(тонни, _, снег), Люди)`

№	Условие	Цель
3	Майкл не любит солнце	`\+ member(человек(майкл, _, солнце), Люди)`
4	Футболист любит снег	`member(человек(_, футболист, снег), Люди)`
5	Тонни не пловец	`\+ member(человек(тонни, пловец, _), Люди)`

Решение. Программа `lab_02/sportsmen.pl` использует метод «образовать и проверить»: `решение/1` перебирает $3! \cdot 3! = 36$ вариантов распределения значений, а `ограничения/1` отбрасывает не удовлетворяющие условиям. Предикат `решить/0` находит все решения, сообщает их число и печатает.

```
% ============================================
% Логическая задача «Спортсмены»
% SWI-Prolog
% --------------------------------------------
% Практическая работа №2, задание 4 (вариант 3),
% «Стили и методы программирования. Часть 2».
%
% Условие. Тонни, Майкл и Джон — спортсмены.
% Один из них боксёр, другой футболист, третий
% пловец. Один любит снег, другой — дождь, третий —
% солнечную погоду.
%   1. Тот, кто любит солнечную погоду, — не боксёр.
%   2. Тонни не любит снег.
%   3. Майкл не любит солнечную погоду.
%   4. Футболист любит снег.
%   5. Тонни не пловец.
% Определить: у кого какая любимая погода и кто
% каким спортом занимается.
%
% Загрузка:  swipl sportsmen.pl
% Запуск:    ?- решить.
% ============================================

% Возможные значения признаков.
виды_спорта([боксёр, футболист, пловец]).
виды_погоды([снег, дождь, солнце]).

% решение(-Люди): Люди — список из трёх термов человек(Имя, Спорт, Погода),
% удовлетворяющий всем условиям задачи.
% Метод «образовать и проверить»: каждому из трёх признаков назначается
% одна из перестановок значений (так значения не повторяются),
% затем проверяются ограничения.
решение(Люди) :-
    Люди = [человек(тонни, С1, П1),
            человек(майкл, С2, П2),
            человек(джон,  С3, П3)],
```

```
    виды_спорта(Спорт),
    виды_погоды(Погода),
    permutation(Спорт,  [С1, С2, С3]),
    permutation(Погода, [П1, П2, П3]),
    ограничения(Люди).

% ограничения(+Люди): условия 1–5 из текста задачи.
ограничения(Люди) :-
    \+ member(человек(_, боксёр, солнце), Люди),     % 1
    \+ member(человек(тонни, _, снег), Люди),        % 2
    \+ member(человек(майкл, _, солнце), Люди),      % 3
    member(человек(_, футболист, снег), Люди),       % 4
    \+ member(человек(тонни, пловец, _), Люди).      % 5

% решить: найти все решения и напечатать их.
решить :-
    findall(Л, решение(Л), Решения),
    length(Решения, N),
    format("Найдено решений: ~d~n", [N]),
    forall(member(Л, Решения), вывести(Л)).

вывести(Люди) :-
    forall(member(человек(Имя, Спорт, Погода), Люди),
           format("~w: ~w, любит ~w~n", [Имя, Спорт, Погода])).
```

Результат показан на рисунке 3.6. Найдено ровно одно решение, значит условия определяют ответ однозначно:

– Тонни — боксёр, любит дождь;
– Майкл — футболист, любит снег;
– Джон — пловец, любит солнечную погоду.

```
lab_02 — swipl sportsmen.pl

artyo@MacBook-Pro lab_02 % swipl sportsmen.pl

1 ?- решить.
Найдено решений: 1
тонни: боксёр, любит дождь
майкл: футболист, любит снег
джон: пловец, любит солнце
true.

2 ?- решение(Люди).
Люди = [человек(тонни, боксёр, дождь), человек(майкл, футболист, снег), человек(джон, пловец, солнце)] ;
false.

3 ?- halt.
```

Рисунок 3.6 – Решение логической задачи: результат `решить/0` и запрос `решение(Люди)`

Проверка рассуждением. Футболист любит снег (условие 4), а Тонни снег не любит (2) и не пловец (5), поэтому Тонни — не футболист и не

пловец, то есть боксёр. Боксёр не любит солнце (1) и снег (снег любит футболист), значит Тонни любит дождь. Майкл не любит солнце (3) и дождь уже занят, поэтому Майкл любит снег и является футболистом, а Джон остаётся пловцом, любящим солнце. Ответ совпадает с результатом программы.

4 АНАЛИЗ РЕЗУЛЬТАТОВ

1. **Динамическая база.** Факты, добавленные `assertz/1`, существуют только в памяти текущего сеанса и исчезают при его завершении; `listing/1` выводит их с пометкой `:- dynamic`. Статические факты `friends.pl` изменять в ходе работы нельзя.
2. **Транзитивность и циклы.** Рекурсивное правило `greater/2` завершается только на ациклических данных, поэтому контроль противоречий вынесен в момент добавления факта. Это дешевле, чем защищать от зацикливания каждый запрос (например, табличным выполнением).
3. **Частичный порядок.** Введённых отношений часто недостаточно для полного упорядочения. Самый большой объект — тот, для которого нет большего, но таких объектов может быть несколько; вывод всех максимальных элементов честнее выбора одного произвольного. Пример из задания именно такой: ответ об одном «самом большом» объекте зависит от того, достаточно ли введённых условий.
4. **Остаточные точки выбора.** Ответ `true ; false.` на запросах `greater(…)`, `largest(X)` и `решение(Люди)` означает, что после найденного решения у системы остались непроверенные альтернативы, но других решений нет. Для `решение/1` это следствие метода «образовать и проверить»: после первого удачного распределения перебор продолжается и подтверждает единственность решения.
5. **Отрицание как неуспех.** Правила `largest/1`, `smallest/1` и ограничения задачи используют `\+`. Оно корректно только при полностью определённых аргументах: в `largest/1` цель `object(X)` стоит перед `\+`, чтобы `X` был связан к моменту проверки.
6. **Кириллица.** SWI-Prolog 10 читает исходные файлы в UTF-8; атомы и предикаты можно писать кириллицей, переменная начинается с заглавной буквы (`Ф`, `Люди`). При печати через `format/2` и в top-level атомы выводятся без кавычек.

5 ВЫВОДЫ

В ходе работы в SWI-Prolog 10.0.2 созданы три программы. Для задания 2 реализована динамическая база фактов, пополняемая вводом с консоли до символа остановки, с проверкой противоречий и правилами для поиска наибольшего и наименьшего объектов; на примере из задания получены самый маленький объект (вишня) и два несравнимых максимальных (яблоко и дуриан), а после добавления недостающего условия — единственный наибольший (дуриан). Для задания 3 записана база знаний «Друзья» варианта 3 и получены ответы на все шесть вопросов. Для задания 4 логическая задача решена методом «образовать и проверить»; найдено единственное решение, совпавшее с ручным рассуждением.

В процессе освоены `assertz/retractall`, объявление `dynamic`, построчный ввод, транзитивные правила, отрицание как неуспех, сбор решений `findall/3` и перестановки `permutation/2`. Серьёзных трудностей не возникло; единственное, что потребовало решения, — возможное зацикливание `greater/2` на противоречивых данных, устранённое проверкой при добавлении факта.

ПРИЛОЖЕНИЕ А. КОНТРОЛЬНЫЕ ВОПРОСЫ ДЛЯ MOODLE

Ниже приведены 30 контрольных вопросов в форме, удобной для визуальной проверки. Исходный банк вопросов для загрузки в Moodle хранится в файле `moodle_questions_gift.txt` в формате GIFT.

1. Для чего служит директива :- dynamic bigger/2. в начале программы?

- ☐ Включает трассировку предиката bigger/2
- ☐ Объявляет, что клаузы bigger/2 можно добавлять и удалять во время выполнения
- ☐ Запрещает изменять факты bigger/2 после загрузки файла
- ☐ Делает предикат bigger/2 видимым из других файлов

Правильный ответ: Объявляет, что клаузы bigger/2 можно добавлять и удалять во время выполнения.

2. Чем assertz/1 отличается от asserta/1?

- ☐ assertz удаляет клаузу, а asserta добавляет
- ☐ assertz добавляет клаузу в конец предиката, asserta — в начало
- ☐ assertz работает только с фактами, asserta — только с правилами
- ☐ Ничем, это синонимы

Правильный ответ: assertz добавляет клаузу в конец предиката, asserta — в начало.

3. Что делает вызов retractall(bigger(_, _))?

- ☐ Удаляет только первый факт bigger/2
- ☐ Удаляет все клаузы bigger/2, при этом сам предикат остаётся определённым
- ☐ Выводит все факты bigger/2 на экран
- ☐ Завершает работу интерпретатора

Правильный ответ: Удаляет все клаузы bigger/2, при этом сам предикат остаётся определённым.

4. Чем retract/1 отличается от retractall/1?

- ☐ retract удаляет первую подходящую клаузу и при возврате — следующие, retractall удаляет все подходящие за один вызов
- ☐ retract удаляет предикат целиком, retractall — только факты
- ☐ retractall можно применять только к статическим предикатам
- ☐ Ничем

Правильный ответ: retract удаляет первую подходящую клаузу и при возврате — следующие, retractall удаляет все подходящие за один вызов.

5. Что вернёт read_line_to_string(user_input, L) при достижении конца ввода (Ctrl+D)?

- ☐ Пустую строку
- ☐ Атом halt
- ☐ Ошибку синтаксиса
- ☐ Атом end_of_file

Правильный ответ: Атом end_of_file.

6. Каков результат split_string(”яблоко > слива , , P)?

- ☐ P = ”яблоко > слива”
- ☐ P = [яблоко, >, слива]
- ☐ P = [”яблоко » ”слива”]
- ☐ false.

Правильный ответ: P = [”яблоко » ”слива”].

7. Для чего в программе используется normalize_space(string(Clean), Line)?

- ☐ Переводит строку в нижний регистр
- ☐ Убирает лишние пробелы в начале, конце и между словами
- ☐ Разбивает строку на слова
- ☐ Преобразует строку в число

Правильный ответ: Убирает лишние пробелы в начале, конце и между словами.

8. Зачем после format(”ввод: ”) вызывают flush_output?

- ☐ Чтобы очистить экран терминала
- ☐ Чтобы закрыть поток вывода
- ☐ Чтобы приглашение без перевода строки немедленно появилось на экране до чтения ввода
- ☐ Чтобы удалить введённые ранее данные

Правильный ответ: Чтобы приглашение без перевода строки немедленно появилось на экране до чтения ввода.

9. Что делает вызов prompt(Old, ”) перед чтением строк из user_input?

- ☐ Запоминает прежнее приглашение в Old и заменяет его пустым, чтобы не выводилось «|: »
- ☐ Запрещает вводить пустые строки
- ☐ Очищает буфер ввода
- ☐ Загружает файл с пустым именем

Правильный ответ: Запоминает прежнее приглашение в Old и заменяет его пустым, чтобы не выводилось «|: ».

10. Зачем у правила greater/2 две клаузы?

- ☐ Вторая клауза нужна для обработки ошибок
- ☐ Первая — базовый случай (прямой факт bigger), вторая — рекурсивный шаг по цепочке отношений
- ☐ Первая работает с числами, вторая — с атомами
- ☐ Две клаузы нужны только для ускорения поиска

Правильный ответ: Первая — базовый случай (прямой факт bigger), вторая — рекурсивный шаг по цепочке отношений.

11. Что произошло бы с greater/2 при цикле в фактах (a больше b, b больше a)?

- ☐ Запрос вернул бы false
- ☐ Интерпретатор отказался бы загружать факты
- ☐ Рекурсия не имела бы конца и привела бы к переполнению стека
- ☐ Результат зависел бы только от порядка ввода

Правильный ответ: Рекурсия не имела бы конца и привела бы к переполнению стека.

12. Какую роль выполняет клауза add_relation(X, Y) :- greater(Y, X), !, ...?

- ☐ Добавляет новый факт в базу
- ☐ Обнаруживает противоречие: Y уже больше X, и отклоняет новый факт
- ☐ Удаляет старые факты
- ☐ Сортирует объекты по размеру

Правильный ответ: Обнаруживает противоречие: Y уже больше X, и отклоняет новый факт.

13. Как читается правило largest(X) :- object(X), \+ bigger(_, X).?

- ☐ X больше любого объекта
- ☐ X — объект, для которого не найдётся ни одного объекта, больший X
- ☐ X — объект, который больше самого себя
- ☐ X — любой объект, имеющий хотя бы один факт bigger

Правильный ответ: X — объект, для которого не найдётся ни одного объекта, больший X.

14. Что означает оператор \+ в Прологе?

- ☐ Арифметическое сложение
- ☐ Логическое отрицание в смысле математической логики
- ☐ Отрицание как неуспех: цель \+ G успешна, если доказать G не удаётся
- ☐ Объединение двух списков

Правильный ответ: Отрицание как неуспех: цель \+ G успешна, если доказать G не удаётся.

15. Что делает предикат findall(X, largest(X), L)?

- ☐ Находит первое решение и останавливается
- ☐ Удаляет все решения largest/1 из базы
- ☐ Собирает в список L значения X по всем решениям largest(X)
- ☐ Проверяет, что largest/1 определён

Правильный ответ: Собирает в список L значения X по всем решениям largest(X).

16. Почему для примера из задания (яблоко, слива, авокадо, папайя, дуриан, вишня) нет единого самого большого объекта?

- ☐ Потому что вводились не все объекты
- ☐ Потому что дуриан меньше папайи
- ☐ Яблоко и дуриан не связаны цепочкой отношений, и ни один из них не доказуемо больше другого
- ☐ Потому что в Прологе нельзя сравнивать более пяти объектов

Правильный ответ: Яблоко и дуриан не связаны цепочкой отношений, и ни один из них не доказуемо больше другого.

17. Какой самый маленький объект в примере с фруктами из задания?

- ☐ слива
- ☐ авокадо
- ☐ вишня
- ☐ яблоко

Правильный ответ: вишня.

18. Для чего служит вызов aggregate_all(count, bigger(_, _), M)?

- ☐ Суммирует значения аргументов bigger/2
- ☐ Подсчитывает число решений цели, то есть число фактов bigger/2
- ☐ Удаляет дубликаты фактов
- ☐ Сортирует факты по возрастанию

Правильный ответ: Подсчитывает число решений цели, то есть число фактов bigger/2.

19. Какой арности предикат день_рождения(иванова, лена, 12, июнь, 1988)?

- ☐ 3
- ☐ 4
- ☐ 5
- ☐ 6

Правильный ответ: 5.

20. Какие люди вернёт запрос `?- день_рождения(Ф, И, _, _, 1988).`?

- ☐ Только Лену Иванову
- ☐ Только Васю Сидоренко
- ☐ Лену Иванову и Машу Сидоренко
- ☐ Всех троих друзей

Правильный ответ: Лену Иванову и Машу Сидоренко.

21. Что означает символ _ в запросе `?- день_рождения(Ф, И, _, май, _).`?

- ☐ Пустое значение, которое обязательно нужно подставить
- ☐ Имя предиката
- ☐ Анонимная переменная: значение на этой позиции не важно и не запоминается
- ☐ Знак вычитания

Правильный ответ: Анонимная переменная: значение на этой позиции не важно и не запоминается.

22. Сколько решений даст запрос `?- хобби(_, маша, Х).`?

- ☐ 0
- ☐ 1
- ☐ 2
- ☐ 3

Правильный ответ: 2.

23. Почему запрос `?- хобби(Ф, И, кино), хобби(Ф, И, спорт).` находит именно Васю?

- ☐ Потому что Вася родился раньше всех
- ☐ Одни и те же переменные Ф и И в обеих целях требуют, чтобы оба хобби принадлежали одному человеку
- ☐ Потому что запятая означает «или»
- ☐ Потому что Вася — единственный друг в базе

Правильный ответ: Одни и те же переменные Ф и И в обеих целях требуют, чтобы оба хобби принадлежали одному человеку.

24. Какое имя в SWI-Prolog является переменной?

- ☐ иванова
- ☐ ’Лена’
- ☐ Ф
- ☐ маша

Правильный ответ: Ф.

25. Как называется метод решения логической задачи в программе sportsmen.pl?

- ☐ Динамическое программирование
- ☐ Метод наименьших квадратов
- ☐ Жадный алгоритм
- ☐ Образовать и проверить (generate and test)

Правильный ответ: Образовать и проверить (generate and test).

26. Сколько различных результатов даёт запрос `?-` findall(P, permutation([a, b, c], P), L), length(L, N).?

- ☐ 3
- ☐ 6
- ☐ 9
- ☐ 27

Правильный ответ: 6.

27. Зачем для видов спорта и для погоды используют permutation/2, а не независимый выбор значений?

- ☐ Для ускорения работы интерпретатора
- ☐ Перестановка гарантирует, что значения признака у троих спортсменов различны и все использованы
- ☐ Чтобы получить значения в алфавитном порядке
- ☐ Это требуется синтаксисом SWI-Prolog

Правильный ответ: Перестановка гарантирует, что значения признака у троих спортсменов различны и все использованы.

28. Какое условие задачи выражает цель \+ member(человек(_, боксёр, солнце), Люди)?

- ☐ Боксёр любит только солнечную погоду
- ☐ Тот, кто любит солнечную погоду, не боксёр
- ☐ Среди спортсменов нет людей
- ☐ Все любят солнечную погоду

Правильный ответ: Тот, кто любит солнечную погоду, не боксёр.

29. Кем является Тонни согласно найденному решению?

- ☐ Футболистом, любит снег
- ☐ Пловцом, любит солнце
- ☐ Футболистом, любит дождь
- ☐ Боксёром, любит дождь

Правильный ответ: Боксёром, любит дождь.

30. Что подтверждает результат «Найдено решений: 1» для логической задачи?

- ☐ Что программа завершилась без ошибок и это всё
- ☐ Что условия задачи определяют ответ однозначно
- ☐ Что в задаче нет ограничений
- ☐ Что программа проверила только один вариант

Правильный ответ: Что условия задачи определяют ответ однозначно.